

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 1 Part A — Uninformed Search Algorithms

Student Name: ___Naima Tahirou

Maiyaki _____

Student ID: _____30252027_____

GitHub Repository Link:

___https://github.com/naimatahiroumaiya/Lab_1_____

Date Submitted: _____6/20/2026_____

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	yes
Estimated percentage of the work influenced by AI	14.46%
Did you attach evidence of AI use where applicable?	yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
ChatGPT	To support learning, understand search algorithms, and troubleshoot coding errors.	Requested explanations of BFS, DFS, DLS, IDS, GridProblem methods, expand function logic, debugging assistance, interpretation of algorithm results, and guidance for reflection questions.	Understanding the lab concepts, implementing the algorithms, debugging code, creating custom maps, and writing interpretations	I reviewed all explanations against the lab requirements, wrote and tested the code in Jupyter Notebook, fixed implementation errors myself, verified outputs using the provided self-checks and result tables, and adapted the final interpretations

				and reflections to reflect my own understanding.

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	yes	n/a
Prompt history	no	n/a
Screenshot of AI interaction	yes	n/a
Exported AI conversation	no	n/a
AI-generated code snippet	yes	n/a
Revised version showing student input	n/a	n/a

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	yes
Any use of AI tools has been clearly declared in this form.	yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	yes
My AI usage does not exceed 25% of the entire work.	yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	yes

Student Signature: _____ NaimaMaiyaki _____

Date: _____ 6/20/2027 _____

Appendixes:

let's start understanding the lab from the beginning

Perfect. Before we write any code, let's understand **what problem this lab is trying to solve**.

Lab 1 Part A: Uninformed Search Algorithms

What is the main idea?

Imagine you have a drone.

The drone is placed on a map.

The map contains:

- Empty spaces where the drone can move
- Obstacles that block movement
- A starting position



Part A

```
from abc import ABC, abstractmethod
from collections import deque
from typing import List, Tuple, Dict, Callable, Optional
import sys
import heapq
import random
import math
import time
```

hi it is not working



Looking at your screenshot, the code itself is fine.

The issue is that the cell has **not been executed yet**.

Notice on the left side of the cell:

[]



If the code had run successfully, you would see something like:

1. reconstruct_path(node)

 Python



 Exécuter

```
def reconstruct_path(node: Node) -> List[Any]:
```

Question:

We found the goal. How do we get the full path from Start → Goal?

Imagine BFS finds:

```
Goal = (2,2)
```



But the goal node stores:

```
(2,2)
Parent → (2,1)
Parent → (2,0)
Parent → (1,0)
Parent → (0,0)
```



Remember:

Each node knows its parent.



+ Poser une question

0



```
def reconstruct_path(node: Node) -> List[Any]:
    """Return the list of states from the root node to this
    node."""
    path = []

    while node is not None:
        path.append(node.state)
        node = node.parent

    path.reverse()
    return path
```

Afficher plus ▾



Excellent. This section is much easier than the previous one.

Think of these functions as **helpers**. They help us work with nodes and display results. We do **not** modify them.



kindly explain the logic
do not provide the code

```
# TODO 2:  
# Return a list of legal action names.  
#  
# Steps:  
# 1. Create an empty list called legal_actions.  
# 2. For each action in MOVES:  
#     a. Compute the neighbour cell.  
#     b. Check that it is in bounds.  
#     c. Check that it is free.
```

Afficher plus ▾

Excellent question. This is exactly the point where many students get stuck because they try to write code before understanding the logic.

Let's imagine we are the drone standing on a cell.

Suppose we are at:

(2,3)



The `actions()` method is asking:

"From where I am right now, what moves am I allowed to make?"

when they said compute Compute the neighbour cell. how is the logic?

This is exactly the right question.

Most students read:

Compute the neighbour cell



and think:

"Where does that neighbour come from?"

Let's build the logic carefully.

What is a neighbour?

A neighbour is simply:

The cell you would reach if you took one move.

Suppose you are standing at:

(2,3)



Think of it as:



! Be careful question

0



```
def result(self, state: Tuple[int, int], action: str) -> Tuple[int, int]:
    # TODO 2:
    # Return the next state after applying action to state.
    #
    # Hint:
    # dr, dc = MOVES[action]
    # return (row + dr, col + dc)
    raise NotImplementedError("Complete GridProblem.result")
```

```
row, col = state
dr, dc = MOVES[action]
next_state = (row + dr, col + dc)
return next_state
```

I tried this code

Yes! Your logic is correct. ✓

You did exactly what the hint was suggesting:

Python

```
row, col = state
dr, dc = MOVES[action]
next_state = (row + dr, col + dc)
return next_state
```



Exécuter

Let's verify it with an example.

Suppose:



```
Traceback (most recent call last):
  File "grid.py", line 9, in <module>
    g = GridProblem(1, 1)
  File "grid.py", line 10, in __init__
    self.grid = list(list(int))
  File "grid.py", line 10, in list
    list(int)
TypeError: 'int' object is not iterable

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "grid.py", line 10, in __init__
    self.grid = list(list(int))
  File "grid.py", line 10, in list
    list(int)
TypeError: 'int' object is not iterable
```

```
MOVES = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1),
}
```

```
class GridProblem(Problem):
    def __init__(
        self,
```

Afficher plus



You're very close. The error is not because your logic is wrong.

Look at the last line of your `actions()` method:



explain this:
5.1 Self-Check for GridProblem
Run this after completing GridProblem.

The test grid is:

```
S . .  
# # .  
. . G
```

Expected behaviour:

Afficher plus ▾

Excellent. This section is not teaching a new concept.

It is simply **testing whether your TODOs are correct**.

Think of it as the lecturer saying:

"Let's see if your `GridProblem` behaves the way it should."

do not provide the code just explain:

8. The Shared expand Method

The pseudocode has a separate EXPAND(problem, node) function.

In this OOP design, all search algorithms inherit a common expand method.

Complete this method once, then reuse it in BFS, DFS, DLS, and IDS.

The method should:

Afficher plus ▾

This section is **the bridge between the problem and the search algorithms**.

Up to now, we have built:

```
Problem  
├─ actions()  
├─ result()  
└─ action_cost()
```



and




```

s = node.STATE
for each action in problem.ACTIONS(s):
    s_prime = problem.RESULT(s, action)
    cost = node.PATH_COST + problem.ACTION_COST(s, action,
s_prime)
    yield = NODE(STATE=s_prime, PARENT=node, ACTION=action,
PATH_COST=cost)

```



You're very close, but there is one important thing:

What you've written is still **pseudocode**, not Python.

Let's focus on the logic only.



I tried the todo7 , check if the logic is correct

```

class DepthFirstSearch(SearchAlgorithm):
    def search(self, problem: Problem) -> SearchResult:
        algorithm = "DFS"
        node = Node(state=problem.initial_state())

        if problem.is_goal(node.state):
            return SearchResult(
                algorithm = algorithm,

Afficher plus ▼

```



The logic is **not correct yet**, but that's okay because you're clearly trying to adapt your BFS solution into DFS.

Let's look at the algorithmic issues rather than syntax.



Good Parts

but it says to reserved

Yes — and that's a very important observation.

The note says:

If you want DFS to explore actions in the same order as MOVES, you may need to reverse the children before pushing them.



Let's see why.

Suppose your `MOVES` dictionary is:

UP
DOWN
LEFT
RIGHT



and `expand()` generates children in exactly that order:

Child 1 = UP
Child 2 = DOWN
Child 3 = LEFT
Child 4 = RIGHT



kindly explain the steps

```
class DepthLimitedSearch(SearchAlgorithm):
```

```
    def search(self, problem: Problem, limit: int = 10) -> SearchResult:
        algorithm = "DLS"
```

```
        initial_node = Node(problem.initial_state())
```

```
        metrics = {
            "nodes_expanded": 0,
            "max_stack_size": 1,
        }
```

Afficher plus ▾

This section is tricky because it uses **recursion**. Let's ignore Python for a moment and think like a person solving a maze.

First: What happens in `search()` ?

The `search()` method is already completed for you.

It does 4 things:

Step 1: Create the start node



```
I tried this logic
if problem.is-goal(node.state):
return SearchResult(
    algorithm=algorithm,
    status=status,
    solution=solution,
    nodes_expanded=metrics["nodes_expanded"],
    max_frontier_size=metrics["max_stack_size"],
    reached_count=0,
    limit=limit,
)
```

Afficher plus ▾

Your thinking is actually quite good. You have the main structure of DLS. The issues are mostly about **what should be returned** and **where some steps belong**.

Let's compare your logic to the required steps.

Step 1: Goal Test

You wrote:



```
if problem.is_goal(node.state)
```

Dictier Ctrl + Shift + D